

Nav2 기본 자율주행 실습

TurtleBot3 · Gazebo · RViz2

cchyun@gmail.com

Part 1. 전체 시스템 구조

- 자율주행 시스템은 여러 독립적인 서버가 협력하여 작동
 - **map_server**
 - 정적 지도(`.yaml`, `.pgm`)를 로드하여 시스템에 제공
 - **amcl**
 - LiDAR 데이터와 지도를 비교하여 로봇의 현재 위치(Pose)를 추정
 - **planner_server (Global Planner)**
 - 현재 위치에서 목적지까지의 최적 경로를 계산 (예: NavFn)
 - **controller_server (Local Planner)**
 - 경로를 따라가며 장애물을 피하기 위한 실시간 속도 명령(`/cmd_vel`)을 생성 (예: DWB)

- **bt_navigator**

- 행동 트리(Behavior Tree)를 사용해 각 서버들의 실행 순서와 예외 상황을 관리
- 외부 클라이언트는 `NavigateToPose` 액션으로 호출

- **TF 체인 (좌표계 연결)**

- `map` → `odom` → `base_link` → `[sensor_frames]` 순으로 연결
- 로봇이 지도 위 어디에 있는지 계산하는 핵심 체인

- **토픽 및 액션 관계**

- 토픽: `/map` (지도), `/scan` (LiDAR), `/tf` (좌표 변환), `/cmd_vel` (속도 명령)
- 액션: `NavigateToPose` (전체 주행 요청)
 - `bt_navigator` 내부에서 `ComputePathToPose` (경로 생성 요청 → `planner_server`)
 - `bt_navigator` 내부에서 `FollowPath` (경로 추종 요청 → `controller_server`)

Part 2. 단계별 구현 및 실행

- 터미널 1에서 시뮬레이션 환경과 지도 기반 위치 추정을 실행

```
cd ~/Workspace/ros2-ws
source install/setup.bash

ros2 launch my_robot_description nav2.launch.py \
  world:=slam_world.world \
  map_yaml:=/home/cchyun/Workspace/ros2_ws/slam_map.yaml
```

- 예상 화면
 - Gazebo 창에 TurtleBot3가 나타남
 - `map_server`와 `amcl`이 실행됨

2단계 — Nav2 서버 실행 및 Lifecycle 관리

- 터미널 2에서 Nav2 내비게이션 서버들을 실행

```
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 launch nav2_bringup navigation_launch.py \
  params_file:=src/my_robot_description/config/nav2_params.yaml \
  use_sim_time:=True
```

- 기동되는 서버

- `planner_server`, `controller_server`, `behavior_server`, `bt_navigator`, `smoother_server`
- 터미널 로그에 "Creating bond timer" 문구가 보이면 lifecycle 활성화 완료

- 주의

- 실행 직후 `global_costmap`에서 `map` frame을 찾지 못한다는 경고가 반복될 수 있음
- 이는 `amcl`이 초기 위치가 설정되기 전까지 `map` frame을 만들지 않기 때문이며 정상 동작

3단계 — RViz2 실행 및 시각화 설정

- 터미널 3에서 RViz2 실행

```
rviz2
```

- RViz2 설정 항목

추가 항목	Topic	설명
Map	/map	Durability: <code>Transient Local</code> 로 설정 필수
Map	/global_costmap/costmap	글로벌 Costmap 시각화
Map	/local_costmap/costmap	로컬 Costmap 시각화
Path	/plan	글로벌 플래너가 생성한 전체 경로
Path	/local_plan	DWB 컨트롤러가 생성한 실시간 궤적
PoseArray	/particlecloud	AMCL 파티클 구름
LaserScan	/scan	LiDAR 데이터
TF	—	좌표계 변환 체인 확인

4단계 — 초기 위치 설정 (2D Pose Estimate)

- 로봇은 처음에 자신의 위치를 모름 → 수동으로 초기 위치 지정 필요

- 절차

- RViz2 상단의 "2D Pose Estimate" 버튼 클릭
- 지도에서 실제 로봇이 있는 위치와 방향을 마우스로 클릭 후 드래그
- 파티클(녹색 화살표)들이 로봇 주변으로 뭉치면 성공

- 확인 명령

```
# AMCL 추정 위치 확인
ros2 topic echo /amcl_pose --once

# TF 트리에 map → odom → base_link 연결 확인
ros2 run tf2_tools view_frames
```

5단계 — 목적지 지정 및 자율주행 (Navigation2 Goal)

- 초기 위치 설정 후 목적지를 지정하여 자율주행 시작
 - 절차
 - RViz2 상단의 "Navigation2 Goal" (또는 2D Nav Goal) 버튼 클릭
 - 지도의 목적지를 클릭하고 방향을 설정
 - 로봇이 경로를 생성하고 주행을 시작
 - 확인 포인트
 - `planner_server`가 `/plan` 생성
 - `controller_server`가 `/local_plan` 실시간 추종
 - 장애물 근처에서는 속도를 줄이거나 회피
 - 목적지 허용 범위 내에 들어오면 주행 완료

6단계 — Python 코드로 호출하기

- `nav2_simple_commander` API를 사용하면 Python으로 주행 제어 가능

```
import time
import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    # 1. Nav2 활성화 대기
    nav.waitUntilNav2Active()

    # 2. 목표 지점 설정
    goal_pose = PoseStamped()
    goal_pose.header.frame_id = 'map'
    goal_pose.pose.position.x = 1.5
    goal_pose.pose.position.y = 0.5
    goal_pose.pose.orientation.w = 1.0

    # 3. 주행 명령
    nav.goToPose(goal_pose)

    # 4. 주행 상태 확인
    while not nav.isTaskComplete():
        feedback = nav.getFeedback()
        if feedback:
            print(f'Distance remaining: {feedback.distance_remaining:.2f}')
            time.sleep(0.1)

    print('목적지에 도착했습니다!')
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Part 3. Nav2 파라미터 가이드

- Costmap은 로봇 주변 장애물 정보를 2D 그리드로 표현한 지도
 - Static Layer
 - 사전에 작성된 지도(`.pgm`) 기반 벽 정보
 - Obstacle Layer (Voxel Layer)
 - 실시간 `/scan` 데이터를 기반으로 동적 장애물 마킹/클리어링
 - Inflation Layer
 - 로봇이 벽에 부딪히지 않도록 장애물 주위에 '안전 거리' 버퍼 생성
 - `inflation_radius` 값이 클수록 더 넓게 회피

- DWB (Dynamic Window Approach) 컨트롤러 — 실시간 궤적 생성

- 속도 제한

- `max_vel_x`: 0.26, `max_vel_theta`: 1.0

- 가속도 제한

- `acc_lim_x`: 2.5, `acc_lim_theta`: 3.2

- Trajectory Critics (평가 가중치)

- `PathAlign.scale`: 32.0 — 경로 정렬 우선순위
- `GoalAlign.scale`: 24.0 — 목적지 방향 정렬
- `BaseObstacle.scale`: 0.02 — 장애물 회피

- 허용 오차

- `xy_goal_tolerance`: 0.25, `yaw_goal_tolerance`: 0.25

파라미터 예시

```
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0          # 코스트맵 업데이트 주기 (Hz) - 1초에 1번 갱신
      publish_frequency: 1.0         # 코스트맵 발행 주기 (Hz) - RViz 등에 1초에 1번 전송
      global_frame: map               # 코스트맵의 기준 좌표계 (map = 전역 지도)
      robot_base_frame: base_link     # 로봇의 베이스 링크 프레임
      use_sim_time: True               # Gazebo 등 시뮬레이션 시간 사용 여부
      robot_radius: 0.22              # 로봇 반경 (m) - 원형 모델 사용 시 장애물과의 여유 거리 계산에 사용
      resolution: 0.05                # 코스트맵 해상도 (m/cell) - 5cm 크기의 한 칸
      track_unknown_space: true        # 미탐색 영역(unknown)을 코스트맵에 유지할지 여부
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"] # 적용할 레이어 목록
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer" # 동적 장애물 감지 레이어
        enabled: True                        # 레이어 활성화 여부
        observation_sources: scan            # 사용할 센서 소스 이름 (아래 scan: 항목과 매칭)
        scan:
          topic: /scan                     # 구독할 토픽 (LaserScan 메시지)
          max_obstacle_height: 2.0          # 장애물로 인식할 최대 높이 (m)
          clearing: True                     # 빈 공간으로 지울지 여부 (raytracing으로 자유공간 표시)
          marking: True                     # 장애물로 표시할지 여부
          data_type: "LaserScan"            # 수신 데이터 타입
          raytrace_max_range: 3.0            # raytracing 최대 거리 (m) - 이 범위 밖은 unknown 처리
          raytrace_min_range: 0.0           # raytracing 최소 거리 (m)
          obstacle_max_range: 2.5           # 장애물로 기록할 최대 거리 (m)
          obstacle_min_range: 0.0           # 장애물로 기록할 최소 거리 (m)
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer" # 맵 파일에서 불러오는 정적 레이어
        map_subscribe_transient_local: True     # 맵을 퍼블리시한 후에도 late subscriber가 받을 수 있도록 설정 (QoS)
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer" # 장애물 주변 여유 공간(inflation) 레이어
        cost_scaling_factor: 3.0              # 비용 감쇠 계수 - 값이 클수록 장애물에서 멀리 갈수록 비용이 빨리 줄어듦
        inflation_radius: 0.55               # 팽창 반경 (m) - 장애물에서 이 거리만큼 여유를 둠
      always_send_full_costmap: True          # 매번 전체 코스트맵을 발행할지 여부 (False면 변경 부분만 발행)
```

파라미터 예시

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0          # 로컬 코스트맵 업데이트 주기 (Hz) - 전역보다 빠르게 갱신
      publish_frequency: 2.0         # 로컬 코스트맵 발행 주기 (Hz)
      global_frame: odom             # 로컬 코스트맵 기준 좌표계 (odom = 로컬 주행 기준)
      robot_base_frame: base_link    # 로봇 베이스 링크 프레임
      use_sim_time: True              # 시뮬레이션 시간 사용
      rolling_window: true            # 로봇 중심으로 창이 이동하는(rolling) 맵 사용 여부
      width: 3                       # 로컬 맵 가로 크기 (m)
      height: 3                      # 로컬 맵 세로 크기 (m)
      resolution: 0.05               # 해상도 (m/cell)
      robot_radius: 0.22             # 로봇 반경 (m)
      plugins: ["voxel_layer", "inflation_layer"] # 적용할 레이어 목록
    inflation_layer:
      plugin: "nav2_costmap_2d::InflationLayer"
      cost_scaling_factor: 3.0        # 비용 감쇠 계수
      inflation_radius: 0.55         # 팽창 반경 (m)
    voxel_layer:
      plugin: "nav2_costmap_2d::VoxelLayer" # 3D 복셀 기반 장애물 레이어 (2D ObstacleLayer의 3D 버전)
      enabled: True                   # 레이어 활성화
      publish_voxel_map: True         # 복셀 맵 시각화 토픽 발행 여부
      origin_z: 0.0                  # 복셀 그리드의 Z축 원점 (m)
      z_resolution: 0.05             # Z축 해상도 (m/cell)
      z_voxels: 16                   # Z축 방향 복셀 개수 (총 높이 = z_resolution × z_voxels = 0.8m)
      max_obstacle_height: 2.0        # 장애물로 인식할 최대 높이 (m)
      mark_threshold: 0               # 복셀이 장애물로 표시되기 위한 최소 점유 횟수
      observation_sources: scan       # 사용할 센서 소스
      scan:
        topic: /scan                 # 구독할 토픽
        max_obstacle_height: 2.0     # 장애물 최대 높이 (m)
        clearing: True                # raytracing으로 자유공간 지우기
        marking: True                 # 장애물 표시
        data_type: "LaserScan"        # 데이터 타입
        raytrace_max_range: 3.0        # raytracing 최대 거리 (m)
        raytrace_min_range: 0.0        # raytracing 최소 거리 (m)
        obstacle_max_range: 2.5        # 장애물 기록 최대 거리 (m)
        obstacle_min_range: 0.0        # 장애물 기록 최소 거리 (m)
      always_send_full_costmap: True  # 전체 로컬 코스트맵 매번 발행
```

파라미터 예시

```
planner_server:
  ros__parameters:
    use_sim_time: True           # 시뮬레이션 시간 사용
    expected_planner_frequency: 20.0 # 기대 플래너 동작 주기 (Hz) - 이 주기보다 늦으면 경고
    planner_plugins: ["GridBased"] # 사용할 플래너 플러그인 목록
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner" # NavFn 경로 계획 알고리즘 플러그인
      tolerance: 0.5 # 목표 지점 허용 오차 (m) - 목표 정확히 못 가면 반경 0.5m 내에서 완료
      use_astar: false # A* 알고리즘 사용 여부 (false = Dijkstra)
      allow_unknown: true # 미탐색(unknown) 영역을 경로로 허용할지 여부
```

파라미터 예시

```
controller_server:
  ros__parameters:
    use_sim_time: True                # 시뮬레이션 시간 사용
    controller_frequency: 20.0        # 컨트롤러 실행 주기 (Hz) - 1초에 20번 속도 명령 계산
    min_x_velocity_threshold: 0.001    # X 방향 최소 속도 임계값 (m/s) - 이 이하면 정지로 간주
    min_y_velocity_threshold: 0.5      # Y 방향 최소 속도 임계값 (m/s)
    min_theta_velocity_threshold: 0.001 # 각속도 최소 임계값 (rad/s)
    failure_tolerance: 0.3             # 컨트롤러 실패 허용 시간 (s) - 이 시간 동안 예외를 참음
    progress_checker_plugin: "progress_checker" # 사용할 진행 체커 플러그인
    goal_checker_plugins: ["general_goal_checker"] # 사용할 목표 체커 플러그인 목록
    controller_plugins: ["FollowPath"] # 사용할 컨트롤러 플러그인 목록

progress_checker:
  plugin: "nav2_controller::SimpleProgressChecker" # 로봇이 실제로 움직이는지 감시
  required_movement_radius: 0.5 # 일정 시간 내 이동해야 하는 최소 반경 (m)
  movement_time_allowance: 10.0 # 이동을 기다리는 최대 시간 (s) - 10초 동안 0.5m 이상 안 움직이면 실패

general_goal_checker:
  stateful: True # 상태 유지 여부 - 한 번 목표 도달하면 계속 도달 상태 유지
  plugin: "nav2_controller::SimpleGoalChecker" # 단순 목표 도달 판정기
  xy_goal_tolerance: 0.25 # 위치 목표 허용 오차 (m)
  yaw_goal_tolerance: 0.25 # 방향(yaw) 목표 허용 오차 (rad)
```

파라미터 예시

```
FollowPath:
  plugin: "dwb_core::DWBLocalPlanner" # DWB (Dynamic Window Approach) 로컬 플래너
  debug_trajectory_details: True      # 디버그 정보 출력 여부
  min_vel_x: 0.0                      # 최소 전진 속도 (m/s)
  min_vel_y: 0.0                      # 최소 횡방향 속도 (m/s)
  max_vel_x: 0.26                     # 최대 전진 속도 (m/s)
  max_vel_y: 0.0                      # 최대 횡방향 속도 (m/s) - 0이면 차동 구동(전진/회전만)
  max_vel_theta: 1.0                 # 최대 각속도 (rad/s)
  min_speed_xy: 0.0                   # XY 평면 최소 속도 (m/s)
  max_speed_xy: 0.26                  # XY 평면 최대 속도 (m/s)
  min_speed_theta: 0.0                # 최소 각속도 (rad/s)
  acc_lim_x: 2.5                      # X 방향 최대 가속도 (m/s²)
  acc_lim_y: 0.0                      # Y 방향 최대 가속도 (m/s²)
  acc_lim_theta: 3.2                  # 각속도 최대 가속도 (rad/s²)
  decel_lim_x: -2.5                   # X 방향 최대 감속도 (m/s²)
  decel_lim_y: 0.0                    # Y 방향 최대 감속도 (m/s²)
  decel_lim_theta: -3.2                # 각속도 최대 감속도 (rad/s²)
  vx_samples: 20                      # X 속도 샘플링 개수 - 후보 속도 중 20개를 샘플링
  vy_samples: 5                       # Y 속도 샘플링 개수
  vtheta_samples: 20                  # 각속도 샘플링 개수
  sim_time: 1.7                       # 궤적 시뮬레이션 시간 (s) - 1.7초 앞을 내다보고 평가
  linear_granularity: 0.05             # 선형 해상도 (m) - 궤적 평가 시 선형 간격
  angular_granularity: 0.025           # 각도 해상도 (rad) - 궤적 평가 시 각도 간격
  transform_tolerance: 0.2             # TF 변환 허용 지연 시간 (s)
  xy_goal_tolerance: 0.25              # 목표 위치 허용 오차 (m) - 컨트롤러 내부용
  trans_stopped_velocity: 0.25         # "멈춤"으로 간주할 선형 속도 임계값 (m/s)
  short_circuit_trajectory_evaluation: True # 궤적 평가 중 불가능한 궤적은 조기 종료 (성능 향상)
  stateful: True                       # 상태 유지 여부
  critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
                                         # 사용할 궤적 평가 크리틱(평가 지표) 목록
  BaseObstacle.scale: 0.02            # 장애물 회피 가중치 (낮음 = 장애물 가까이 가도 됨)
  PathAlign.scale: 32.0                # 경로 방향 정렬 가중치 (높음 = 경로 방향을 잘 따름)
  PathAlign.forward_point_distance: 0.1 # 경로 정렬 시 앞을 볼 거리 (m)
  GoalAlign.scale: 24.0                # 목표 방향 정렬 가중치
  GoalAlign.forward_point_distance: 0.1 # 목표 정렬 시 앞을 볼 거리 (m)
  PathDist.scale: 32.0                 # 경로 거리 유지 가중치 (높음 = 경로를 가까이 따름)
  GoalDist.scale: 24.0                 # 목표 거리 가중치 (높음 = 목표에 빨리 가까워짐)
  RotateToGoal.scale: 32.0             # 목표에서 회전 가중치
  RotateToGoal.slowing_factor: 5.0     # 목표 근처에서 속도를 줄이는 계수
  RotateToGoal.lookahead_time: -1.0    # 목표 회전 시 선행 시간 (-1 = 비활성화)
```

파라미터 예시

```
behavior_server:
  ros__parameters:
    costmap_topic: local_costmap/costmap_raw          # 참조할 코스트맵 토픽
    footprint_topic: local_costmap/published_footprint # 참조할 풋프린트 토픽
    cycle_frequency: 10.0                             # Behavior 서버 실행 주기 (Hz)
    behavior_plugins: ["spin", "backup", "drive_on_heading", "assisted_teleop", "wait"]
                                                         # 사용할 행동(복구) 플러그인 목록

  spin:
    plugin: "nav2_behaviors/Spin"                     # 제자리 회전 동작
  backup:
    plugin: "nav2_behaviors/BackUp"                   # 후진 동작
  drive_on_heading:
    plugin: "nav2_behaviors/DriveOnHeading"           # 정면 방향으로 직진 동작
  wait:
    plugin: "nav2_behaviors/Wait"                     # 대기 동작
  assisted_teleop:
    plugin: "nav2_behaviors/AssistedTeleop"          # 보조 텔레오퍼레이션

  global_frame: odom                                  # 동작 수행 기준 좌표계
  robot_base_frame: base_link                        # 로봇 베이스 프레임
  transform_tolerance: 0.1                           # TF 변환 허용 지연 시간 (s)
  use_sim_time: true                                  # 시뮬레이션 시간 사용
  simulate_ahead_time: 2.0                           # 동작 실행 전 충돌 예측 시뮬레이션 시간 (s)
  max_rotational_vel: 1.0                             # 최대 회전 속도 (rad/s)
  min_rotational_vel: 0.4                             # 최소 회전 속도 (rad/s)
  rotational_acc_lim: 3.2                             # 회전 가속도 제한 (rad/s2)
```

Part 4. 자주 발생하는 오류와 해결법

- 오류 1: Lifecycle 활성화 실패

- 증상: 터미널 로그에 "Creating bond ***" 문구가 나오지 않음
- 원인: 노드 간 실행 순서 문제 또는 다른 노드 미기동
- 해결: 모든 서버가 기동된 후 재시도. 터미널 로그에 "Creating bond ???"가 보이는지 확인

- 오류 2: RViz에서 지도가 안 보임

- 증상: `/map` 토픽을 추가해도 지도가 표시되지 않음
- 원인: QoS 설정 오류
- 해결: `Map` 설정에서 `Durability Policy`를 `Transient Local`로 변경

- 오류 3: Costmap 미업데이트

- 증상: RViz에서 Costmap이 갱신되지 않음
- 원인: 센서 토픽명이 설정 파일과 불일치
- 해결: `nav2_params.yaml`에서 `topic: /scan`이 실제 토픽과 일치하는지 확인

- 오류 4: 경로 계획 실패 (No Path Found)

- 증상: 주행 명령 후 경로가 생성되지 않음
- 원인: 목적지가 장애물 내부 또는 지도 밖
- 해결: 빈 공간(흰색 영역)을 목적지로 다시 지정. `allow_unknown: true` 확인

- 오류 5: 로봇이 제자리에서 회전만 함

- 증상: 주행 시작 직후 로봇이 계속 제자리 회전
- 원인: 초기 방향 설정이 크게 어긋났을 때 발생
- 해결: `2D Pose Estimate`로 초기 위치 재설정.

- 오류 6: 벽에 너무 가깝게 주행

- 증상: 로봇이 장애물 근처를 위험하게 지나감
- 원인: `inflation_radius`가 너무 작음
- 해결: `inflation_radius`를 키우거나 `cost_scaling_factor`를 낮춤

- 문제 해결에 활용할 수 있는 필수 명령어

```
ros2 topic list           # 현재 발행 중인 토픽 목록
ros2 topic echo /amcl_pose --once  # AMCL 추정 위치 확인
ros2 topic echo /plan --once      # 글로벌 경로 확인
ros2 topic hz /scan           # LiDAR 데이터 수신 주기
ros2 node list             # 동작 중인 노드 확인
ros2 run tf2_tools view_frames    # TF 트리 시각화 (PDF 생성)
ros2 run rqt_plot rqt_plot /cmd_vel/linear/x  # 선속도 그래프
ros2 run rqt_plot rqt_plot /cmd_vel/angular/z  # 각속도 그래프
```

Part 5. 심화 미션

미션 1 — rqt_plot으로 속도 모니터링

- 로봇이 가속하고 감속하는 과정을 그래프로 실시간 확인

- 실행 명령어

```
# 선속도 모니터링
ros2 run rqt_plot rqt_plot /cmd_vel/linear/x

# 각속도 모니터링
ros2 run rqt_plot rqt_plot /cmd_vel/angular/z
```

- 관찰 포인트

- DWB 컨트롤러가 장애물 근처에서 속도를 줄이는지 확인
- 목적지 접근 시 감속 패턴 관찰

미션 2 — 순차 이동 미션 (NavigateThroughPoses)

- **NavigateThroughPoses** 액션을 이용한 웨이포인트 네비게이션

- **미션 목표**

- 1번 지역 → 2번 지역 → 3번 지역 순으로 이동

- **예시 좌표 (map 프레임 기준)**

지점	x	y	yaw
1번	1.0	1.0	0.0
2번	3.5	-1.0	1.57
3번	0.0	0.0	0.0 (복귀)

- **확인 포인트**

- 각 지점에 도달 후 다음 지점으로 전환되는지 확인
- 경로 중간에 장애물이 추가될 경우 재계획 후 우회하는지 관찰



```
import time
import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped
from math import sin, cos

def main():
    rclpy.init()
    nav = BasicNavigator()
    nav.waitUntilNav2Active()

    waypoints = [(1.0, 1.0, 0.0), (3.5, -1.0, 1.57), (0.0, 0.0, 0.0)]
    poses = []

    for x, y, yaw in waypoints:
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.orientation.z = sin(yaw / 2.0)
        pose.pose.orientation.w = cos(yaw / 2.0)
        poses.append(pose)

    nav.followWaypoints(poses)

    while not nav.isTaskComplete():
        feedback = nav.getFeedback()
        if feedback:
            print(f'Current waypoint: {feedback.current_waypoint}')
            time.sleep(0.1)

    print('Mission complete!')
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```